

Exame de época normal: engweb2026-normal

UC: Engenharia Web (3º ano LEI)

Data: 17 de Julho de 2026, 9h, Ed. 2 - sala 0.03

Sinopsis

O objectivo principal deste teste é avaliar os conhecimentos obtidos durante as aulas no desenvolvimento de aplicações Web e outras tarefas afins.

Antes de começares, lê atentamente até ao fim para ficares com uma percepção do todo que se pretende.

Vais ver que tomarás decisões mais acertadas depois de uma leitura completa.

Os resultados finais deverão ser enviados ao docente da seguinte forma:

- Enviar email para: `jcr@di.uminho.pt`;
- Colocar no subject/assunto: `ENGWEB2026::Especial::Axxxxx`, em que `Axxxxx` corresponde ao número do aluno;
- Enviar ao docente um link do github para um repositório novo criado especificamente para o exame com o seguinte conteúdo (esta preparação poderá valer 1 valor do exame):
 - O repositório no GitHub deverá chamar-se `ENGWEB2026-Especial`;
 - Dentro do repositório deverá haver um ficheiro, `README.md`, contendo uma descrição de como fez a persistência de dados, do setup de bases de dados, respostas textuais pedidas, instruções de como executar as aplicações desenvolvidas, etc. As instruções deverão ser necessárias e suficientes para o docente, a partir do material no repositório, conseguir colocar em execução todos os serviços e testar os requisitos pedidos;
 - Dentro do repositório deverão existir duas pastas: `ex1`, onde colocarão a aplicação desenvolvida para responder ao primeiro exercício e `ex2`, onde colocarão a aplicação desenvolvida para responder ao segundo exercício.

Os exercícios que envolvam criação de rotas serão testados com as rotas no enunciado, qualquer rota que seja diferente da pedida será avaliada com 0.

Recursos

Recursos para a realização da prova:

- Base de dados sobre Óperas (dataset gerado pela unificação de compositores, cantores, árias, teatros e óperas). Este ficheiro (`operas.json`) tem a seguinte estrutura unificada:

```
[
  {
    "id": "don-giovanni",
    "title": "Don Giovanni",
```

```
"genre": "Opera buffa",
"premiereYear": 1787,
"runtimeMinutes": 170,
"descriptionEN": "Based on the legends of Don Juan, a fictional
libertine and seducer, blending comedy, melodrama and supernatural
elements.",
"compositor": {
  "id": "wolfgang-amadeus-mozart",
  "name": "Wolfgang Amadeus Mozart"
},
"teatro": {
  "id": "estates-theatre",
  "name": "Estates Theatre",
  "country": "Czech Republic"
},
"arias": [
  {
    "id": "madamina-il-catalogo-e-questo",
    "name": "Madamina, il catalogo è questo",
    "voiceType": "Bass-baritone"
  }
],
"cantores": [
  "Joan Sutherland",
  "Anna Netrebko",
  "Bryn Terfel",
  "Renée Fleming",
  "Dietrich Fischer-Dieskau",
  "Tito Gobbi",
  "Cecilia Bartoli",
  "Leontyne Price"
]
},
{
  "id": "la-boheme",
  "title": "La Bohème",
  "genre": "Opera",
  "premiereYear": 1896,
  "runtimeMinutes": 115,
  "descriptionEN": "The tragic love story of the poet Rodolfo and the
seamstress Mimì in 1840s Paris.",
  "compositor": {
    "id": "giacomo-puccini",
    "name": "Giacomo Puccini"
  },
  "teatro": {
    "id": "teatro-regio-turin",
    "name": "Teatro Regio di Torino",
    "country": "Italy"
  },
  "arias": [
    {
      "id": "che-gelida-manina",
      "name": "Che gelida manina",
```

```
    "voiceType": "Tenor"
  },
],
"cantores": [
  "Luciano Pavarotti",
  "Anna Netrebko",
  "Enrico Caruso"
],
},
...
]
```

Exercício 1: O Mundo da Ópera (API de dados)

Neste exercício, irás implementar uma API de dados sobre o dataset unificado a partir de vários pequenos datasets fornecidos em JSON. Encontra-se dividido em 3 partes.

1.1 Setup [1 val.]

Realiza as seguintes tarefas seguindo as sugestões para os nomes da base de dados e da(s) coleção(s) fornecidos:

- Cria o ficheiro JSON unificado final a partir da estrutura de exemplo mostrada acima;
- Importa-o numa base de dados em MongoDB com os seguintes parâmetros:
 - **database**: deverá chamar-se **operas_db**;
 - **collection**: deverá haver pelo menos uma de nome **operas**, poderás criar mais se achares que vai facilitar a gestão da informação.
- Testa se a importação correu bem.

1.2 Queries (warm-up) [0.5 + 0.5 + 1 + 1 + 1 = 4 val.]

Especifica queries em MongoDB para responder às seguintes questões:

- Quantas óperas estão registadas na base de dados?
- Quantas óperas têm uma duração (runtimeMinutes) superior a 150 minutos?
- Qual a lista de países onde ocorreram estreias (ordenada alfabeticamente e sem repetições)?
- Qual a distribuição de óperas por compositor (quantas óperas cada compositor tem registadas)?
- Quais as óperas que estrearam em Itália? Devolve apenas o título e o ano de estreia.
- Regista estas queries num ficheiro de texto que deverás colocar na pasta ex1 chamado **queries.txt**, ou acrescenta-as no ficheiro **README.md**.

1.3 API de dados [0.5 + 0.5 + 1 + 1 + 1 + 1 + 1 + 0.5 + 1 + 0.5 = 8 val.]

Nas rotas pedidas a seguir, é sugerido um determinado formato para a resposta. Se achares útil podes aumentar a informação que é devolvida. As rotas serão testadas tal como constam deste enunciado.

Desenvolve uma API de dados, que responde na porta **17060** e que responda às seguintes rotas/pedidos:

- **GET /operas**: devolve uma lista com todas as óperas (campos: **id** (ou **_id**), **title**, **premiereYear**, **compositor.name**, **teatro.country**);
- **GET /operas/:id**: devolve toda a informação da ópera com o identificador passado na rota (todos os campos);
- **GET /operas?comp=CCCC**: devolve a lista de óperas que compostas pelo compositor com identificador CCCC (ex: "giacomo-puccini"): **id** (ou **_id**), **title**, **premiereYear**;
- **GET /cantores**: devolve a lista dos cantores presentes no dataset, ordenada alfabeticamente por nome e sem repetições (lista de pares: nome do cantor, lista de óperas em que cantou, cada ópera representada por um par: id, título);
- **GET /compositores**: devolve a lista dos compositores, ordenada alfabeticamente e sem repetições (lista com: identificador e nome do compositor, e lista de óperas que lá estrearam, para cada ópera, o seu identificador e título);
- **POST /operas**: acrescenta um registo novo à BD, neste caso, uma nova ópera;
- **DELETE /operas/:id**: elimina da BD o registo correspondente à ópera com o identificador passado na rota;
- **PUT /operas/:id**: altera o registo da ópera com o identificador passado na rota;
- Acrescenta uma interface swagger à tua API de dados;
- Cria a(s) Dockerfile(s) necessária(s) e orquestra tudo num docker compose.

Antes de prosseguires, testa as rotas realizadas com o Postman, ou a tua interface swagger ou similar.

Requisitos para o Exercício 2: API de dados ou json-server

Se por algum motivo não conseguiste obter uma API de dados a funcionar que te permita continuar a realizar este teste, podes avançar para o exercício 2 colocando o dataset num **json-server** e usando este como API de dados (terá uma valorização de 20%, ou seja, há uma redução de 80% relativamente à valorização do exercício 1.3).

Se optares por esta via, coloca o **json-server** a responder na **porta 17060**.

Exercício 2: Ópera (Interface) [1+2+2+2 = 7val.]

Tendo a API desenvolvida, desenvolve agora um novo serviço, que responde na **porta 17061** da seguinte forma:

1. Se colocares no browser o endereço **http://localhost:17061** deverás obter a página principal constituída por:
 - Um cabeçalho com metainformação à tua escolha;
 - Uma tabela contendo a lista de registos, um por linha, com os campos: **id** (ou **_id**), **title**, **premiereYear**, **compositor.name**, **teatro.country**, **teatro.name**;
 - O campo **id** deverá ser um link para a página da ópera com esse identificador;
 - O campo **teatro.name** deverá ser um link para a página desse teatro;
 - O campo **compositor.name** deverá ser um link para a página desse compositor.
2. Se colocares no browser o endereço **http://localhost:17061/:id** deverás obter a página da ópera cujo identificador foi passado na rota:

- Esta página deverá conter toda a informação da ópera:
 - identificador, ano de estreia;
 - informação do teatro e do compositor;
 - uma tabela com a lista de cantores (todos os campos de cantor);
 - uma tabela com a lista de arias (todos os campos de cada aria);
 - o compositor deverá ser um link para a página do compositor;
 - e um link para voltar à página principal.
 - 3. Se colocares no browser o endereço `http://localhost:17061/teatros/:id` deverás obter a página do teatro cujo identificador corresponde ao parâmetro passado na rota:
 - Na página de cada teatro deverá constar o seu nome;
 - Uma tabela com a lista das óperas que aí fizeram a sua estreia (`id` ou `_id`, `title`, `premiereYear`, `compositor.name`);
 - Todos os ids de ópera devem ser links para a respetiva página individual da ópera;
 - E um link para voltar à página principal.
 - 4. Se colocares no browser o endereço `http://localhost:17061/compositores/:id` deverás obter a página do compositor cujo identificador corresponde ao parâmetro passado na rota:
 - Na página de cada compositor deverá constar o seu nome;
 - Uma tabela com a lista das óperas que compôs (para cada ópera: `id` (ou `_id`), `title`, `premiereYear`, `teatro.country`, `teatro.name`);
 - Todos os ids de ópera devem ser links para a respetiva página individual da ópera;
 - O `teatro.name` deverá também ser um link para a página individual do teatro;
 - E um link para voltar à página principal.
 - 5. Todas as páginas web deverão ter um rodapé onde deverá constar a data do dia atual e o seguinte texto "Gerado por MyOpera desenvolvido no contexto de EngWeb2026";
 - 6. Preparar o ambiente para distribuição com Docker:
 - Criar as especificações Dockerfile necessárias;
 - Criar um ficheiro `docker-compose.yml` final que orquestre os serviços (esta orquestração deverá também conter a API de dados).
-

Bom trabalho e boa sorte, jcr